

effect. After processing this LaTeX file, you will get an output document, which will show the formatted text.

Usually, on execution, LaTeX files produce PDF (Portable Document Format) or DVI (device independent file format) files as output. Consider the simple LaTeX script 'Hello.tex' shown below, containing only the absolutely essential features of LaTeX. This LaTeX script and all the other PGF/TikZ scripts discussed in this article can be downloaded from opensourceforu.com/article_source_code/May18graphicsvisualisation.zip.

```
\documentclass{article}
\begin{document}
  \textbf{\textit{Hello World}}
\end{document}
```

From the name *Hello.tex* itself it is clear that the extension of a LaTeX file is *tex*. We use this LaTeX file as a template for executing all the PGF/TikZ code discussed in this article. First, let us go through this minimal LaTeX example, line by line. The line of code `\documentclass{article}` defines the document class as an article. Some other document classes provided by LaTeX include letter, book, report, beamer, etc. I believe that except for the document class beamer, which is used to create presentations using LaTeX, the purpose of all the other document classes mentioned here is clear from their names itself. The lines of code `\begin{document}` and `\end{document}` define the area in which the actual data of the output document can be given. In this case, we only have one line of code in the content part of the LaTeX document, `\textbf{\textit{Hello World}}`. This line tells the LaTeX interpreter to print the text 'Hello World' in bold and italics.

So, now we have a LaTeX file ready with us to be executed, but what is the best way to process it? There are different ways to process LaTeX files. For example, you can use Overleaf, a collaborative writing and publishing system that can process LaTeX code. But, in this article, I will be exploring two other methods to process LaTeX.

First, let's execute the LaTeX interpreter from the terminal. Open a terminal in the directory containing the LaTeX file *Hello.tex* and execute the command `latex Hello.tex`. On execution, the LaTeX file will produce a document called *Hello.dvi* with the text 'Hello World' printed in bold and italics. The default output format of LaTeX is DVI. If you want the output file in PDF format, execute the following command on the terminal `pdflatex Hello.tex`, which will use an extension of LaTeX called pdfLaTeX to produce the output file *Hello.pdf* with the same content as the other output file *Hello.dvi*.

The second method is to use a LaTeX editor like Texmaker, a popular free and open source cross-platform LaTeX editor with an integrated PDF and DVI viewer. Open the LaTeX file *Hello.tex* in Texmaker and, in the menu, you will find *Tools > LaTeX* and *Tools > PDFLaTeX* to execute

LaTeX and its extension pdfLaTeX. Whether you use the tool LaTeX, pdfLaTeX or Texmaker, the formatted text in the output document will be the same. Even though Texmaker has the built-in option to view PDF and DVI output files, make sure that your system has a document viewer like Okular that is capable of displaying DVI and PDF files so that the documents can be viewed directly. There are many other LaTeX editors like TeXstudio, TeXworks, Kile, etc. Using Texmaker is just a personal choice, and I encourage you to try out any of the other LaTeX editors that are available and then stick with the one you like the most. Also, remember that sometimes you need to process the LaTeX file multiple times to obtain the correct labels, bibliography, index, etc.

One problem that often confuses new users of LaTeX is the large number of supporting files generated by it. If you observe the directory containing the file *Hello.tex*, you will see files called *Hello.aux* and *Hello.log*, apart from the output files *Hello.pdf* and *Hello.dvi*. And as your LaTeX document becomes more and more complex, the number of such files created also increases. I am not going to explain the purpose of these supporting files in this article, but you don't need to worry because out of the large number of files created, you only need to worry about the LaTeX file with the extension *tex* and the output files with extensions *pdf* and *dvi*. All other files can be regenerated by executing the LaTeX interpreter.

An introduction to PGF/TikZ

Now that we are familiar with how LaTeX works, let us talk about PGF/TikZ, which was developed by Till Tantau, using the software, TeX. As mentioned earlier, it is not a single tool but two separate languages paired together to produce high quality vector graphics. PGF is the lower-level language and TikZ is a set of higher-level macros that use PGF. The relationship between PGF and TikZ is somewhat similar to the relationship between TeX and LaTeX, the former stating the basic syntax and the latter providing macros for ease of use.

TikZ was first introduced with version 0.95 of PGF. The latest version of PGF/TikZ is 3.0.1 released in August 2015. PGF is the acronym for 'portable graphics format'. TikZ is a recursive acronym for 'TikZ ist kein Zeichenprogramm', German for 'TikZ is not a drawing program' — asserting the fact that these are programming languages capable of powerful functions. Unlike the other popular graphics visualisation tool called PSTricks, which uses a syntax resembling PostScript, PGF and TikZ describe the images to be drawn by using a syntax that resembles MetaPost, a language that is used to produce vector graphic diagrams.

This over-dependence of PSTricks on PostScript is an advantage and disadvantage at the same time. The disadvantages of PSTricks include the limited output formats it offers (PSTricks can only produce PostScript output and not PDF output) and incompatibility with some associated tools of LaTeX like pdfTeX. But PGF/TikZ is able to produce both PostScript and PDF output and has a high level of